MIMII Acoustic Anomaly Detection

GitHub Repository Structure & Step-by-Step Setup Guide

Edge Deployment on SAMA7G54-EK | Fan 6dB | Gaussian + Hitachi TFLite

1. Repository Structure

The recommended GitHub repository layout for this project:

```
mimii-anomaly-detection/
├── README.md
├── requirements.txt
├── .gitignore
│
├── Host/                           # All PC-side scripts
│   ├── data_loader.py            # Feature extraction (librosa-based)
│   ├── train_gaussian.py         # Fit Gaussian model → .npy files
│   ├── train_hitachi.py          # Train autoencoder → .h5 + .tflite
│   └── evaluate.py              # AUC comparison on PC
│
├── Board/                        # All board-side scripts
│   ├── infer_gaussian.py         # Pure numpy Gaussian inference
│   └── infer_hitachi.py          # TFLite Hitachi inference
│
├── models/                       # Trained model artifacts (git-ignored)
│   ├── gaussian_mu_id_00.npy
│   ├── gaussian_sigma_inv_id_00.npy
│   ├── gaussian_mu_id_02.npy
│   ├── gaussian_sigma_inv_id_02.npy
│   ├── gaussian_mu_id_04.npy
│   ├── gaussian_sigma_inv_id_04.npy
│   ├── gaussian_mu_id_06.npy
│   ├── gaussian_sigma_inv_id_06.npy
│   ├── hitachi_id_00.tflite
│   ├── hitachi_id_02.tflite
│   ├── hitachi_id_04.tflite
│   └── hitachi_id_06.tflite
│
├──
│── numpy-1.24.4-cp38-cp38-linux_armv7l.whl
│── libopenblas.so.0
│── libgfortran.so.5
│── libgcc_s.so.1
│── libgomp.so.1
│── libstdc++.so.6
│
│
└── setup_guide.docx         # This document
```

1.1 requirements.txt

```
numpy
librosa
scikit-learn
tensorflow==2.8.0
soundfile
```

```
matplotlib
protobuf==3.20.3
```

2. PC Setup (Windows)

Step 1 — Install pyenv-win

Open PowerShell as Administrator and run:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
Invoke-WebRequest -UseBasicParsing -Uri
"https://raw.githubusercontent.com/pyenv-win/pyenv-win/master/pyenv-
win/install-pyenv-win.ps1" -OutFile "./install-pyenv-win.ps1"
&"./install-pyenv-win.ps1"
```

Close and reopen PowerShell. Add pyenv to PATH if not automatic:

```
[System.Environment]::SetEnvironmentVariable("PYENV",
"$env:USERPROFILE\.pyenv\pyenv-win", "User")
$path = [System.Environment]::GetEnvironmentVariable("PATH", "User")
[System.Environment]::SetEnvironmentVariable("PATH",
"$env:USERPROFILE\.pyenv\pyenv-win\bin;$env:USERPROFILE\.pyenv\pyenv-
win\shims;$path", "User")
```

Verify:

```
pyenv --version   # should print pyenv 3.x.x
```

> ⚠ If pyenv command not found after reopening PowerShell, manually add the bin and shims paths above to your user PATH via System Properties → Environment Variables.

Step 2 — Install Python 3.8.10

```
pyenv install 3.8.10
```

> ⚠ Python 3.8 is required — must match the board's Python version. Do not use 3.9, 3.10, 3.11, or 3.12.

Step 3 — Create Project & Virtual Environment

```
cd C:\Users\<your_username>
mkdir mimii_project
cd mimii_project
pyenv local 3.8.10
C:\Users\<your_username>\.pyenv\pyenv-win\versions\3.8.10\python.exe -m venv
venv
venv\Scripts\activate
python --version   # must show Python 3.8.10
```

> ⚠ If python --version shows 3.12 instead of 3.8, the venv picked up the global Python. Deactivate and use the full pyenv path as shown above.

Step 4 — Install Packages

```
pip install numpy librosa scikit-learn tensorflow==2.8.0 soundfile matplotlib
pip install protobuf==3.20.3
```

> ⚠ The protobuf downgrade is mandatory. Without it, TensorFlow 2.8 throws a Descriptors error on import.

Verify:

```
python -c "import numpy, librosa, sklearn, tensorflow, soundfile, matplotlib;
print('all good')"
```

Step 5 — Dataset

Download the MIMII dataset from Zenodo. For this project only the fan/6dB subset is needed. Place it at:

```
C:\Users\<your_username>\Desktop\6_dB_fan\fan\
```

Expected structure:

```
fan/
├── id_00/
│   ├── normal/      (~1000 WAV files)
│   └── abnormal/    (~350-400 WAV files)
├── id_02/ ...
├── id_04/ ...
└── id_06/ ...
```

> ⓘ Update DATA_ROOT in data_loader.py to point to your actual dataset path.

Step 6 — Edit data_loader.py

Open pc/data_loader.py and set:

```
DATA_ROOT = r"C:\Users\<your_username>\Desktop\6_dB_fan\fan"
```

Step 7 — Verify Data Loader

```
python pc/data_loader.py
```

Expected output:

```
id_00 — train: 505, test normal: 506, test abnormal: 407
Gaussian feature shape: (68,)
Hitachi feature shape:  (309, 320)
```

3. Training

Step 8 — Train Gaussian Model

```
python pc/train_gaussian.py
```

Output: 8 .npy files in models/ (mu + sigma_inv per ID). Completes in under 1 minute.

Step 9 — Train Hitachi Autoencoder

```
python pc/train_hitachi.py
```

Output: 4 .h5 files and 4 .tflite files in models/. Takes 10-20 minutes (50 epochs × 4 IDs).

> ⓘ The .tflite files are what get transferred to the board. The .h5 files are kept as backup for re-conversion.

Step 10 — Evaluate on PC

```
python pc/evaluate.py
```

Expected output:

```
ID          Gaussian AUC  G time(ms)   Hitachi AUC  H time(ms)
------------------------------------------------------------
id_00            0.9348       17.32        0.7712       37.16
id_02            0.9977       17.36        0.9882       42.68
id_04            0.9767       20.71        0.9412       42.49
id_06            0.9999       19.60        0.9947       42.89
------------------------------------------------------------
Macro avg         0.9773                     0.9238
```

4. Board Setup (SAMA7G54-EK)

Step 11 — Flash SD Card

Download the Linux4SAM 2022.04 headless image for SAMA7G5-EK:

```
SAMA7G5-EK-linux4sam-poky-sama7g5ek-headless-2022.04.img.bz2
```

From: https://www.linux4sam.org → SAMA7G5-EK demo images → 2022.04 archive

Flash to SD card using Balena Etcher (https://etcher.balena.io). Etcher handles .bz2 directly.

Step 12 — Create /data Partition

Before inserting SD card into board, use DiskGenius (Windows) to create a new ext4 partition in the unallocated space (28+ GB). This becomes the /data working partition.

> ⚠ Do NOT resize the existing root partition (partition 2, ~533MB). Only create a NEW partition in unallocated space. Resizing root will break the bootloader.

Step 13 — First Boot

Insert SD card, connect Ethernet, connect USB serial (COM port). Open PuTTY:

- Connection type: Serial
- Serial line: COMx (check Device Manager for J-Link CDC port)
- Speed: 115200

Power on. Login as root (no password). Set date and get network:

```
date -s "2026-03-24 09:00:00"
udhcpc -i eth0
ping -c 2 google.com
```

> ⚠ The board clock resets to 2018 on every reboot. Always run date -s before pip commands or SSL verification will fail.

Step 14 — Mount /data Partition

```
mkdir -p /data
mount /dev/mmcblk1p3 /data
echo '/dev/mmcblk1p3 /data ext4 defaults 0 0' >> /etc/fstab
df -h /data   # should show ~27GB available
```

> ⓘ Your SD card partition number may differ. Run lsblk to confirm the correct device name.

Step 15 — Install tflite-runtime

```
pip3 install tflite-runtime==2.11.0 --no-deps
```

> ⚠ Always use --no-deps. Without it, pip tries to build numpy from source which fails (no gcc on board).

Step 16 — Cross-Compile numpy on PC

On your PC (requires Docker Desktop with WSL2):

```
docker run --rm --platform linux/arm/v7 -v C:\path\to\wheels:/wheels
python:3.8-bullseye bash -c \
  "apt-get update -q && apt-get install -y -q gcc g++ gfortran libopenblas-dev
&& \
   pip wheel numpy --no-deps -w /wheels -q && \
   cp /usr/lib/arm-linux-gnueabihf/libopenblas*.so* /wheels/ && \
   cp /usr/lib/arm-linux-gnueabihf/libgfortran*.so* /wheels/ && \
   cp /usr/lib/gcc/arm-linux-gnueabihf/10/libgcc_s.so /wheels/ && \
   cp /usr/lib/arm-linux-gnueabihf/libgomp*.so* /wheels/ && \
   cp /usr/lib/arm-linux-gnueabihf/libstdc++*.so* /wheels/ && echo DONE"
```

> ⚠ Must use python:3.8-bullseye (glibc 2.31) not python:3.8-slim/bookworm (glibc 2.34). The board has glibc 2.31 and wheels built against newer glibc will fail to load.

Step 17 — Setup SSH Key (Passwordless Transfer)

On PC:

```
ssh-keygen -t ed25519 -f C:\Users\<user>\.ssh\sama7g54 -N ""
Get-Content C:\Users\<user>\.ssh\sama7g54.pub | ssh root@192.168.100.46 "mkdir
-p ~/.ssh && cat >> ~/.ssh/authorized_keys"
```

On board:

```
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

Test:

```
ssh -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes root@192.168.100.46
"echo ok"
```

Step 18 — Transfer numpy Wheel and Libraries

From PC PowerShell:

```
scp -O -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes wheels\numpy-
1.24.4-cp38-cp38-linux_armv7l.whl root@192.168.100.46:/data/
scp -O -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes
wheels\libopenblas.so.0 root@192.168.100.46:/usr/lib/libopenblas.so.0
scp -O -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes
wheels\libgfortran.so.5 root@192.168.100.46:/usr/lib/libgfortran.so.5
scp -O -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes
wheels\libgcc_s.so root@192.168.100.46:/usr/lib/libgcc_s.so.1
```

```
scp -O -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes
wheels\libgomp.so.1 root@192.168.100.46:/usr/lib/libgomp.so.1
scp -O -i C:\Users\<user>\.ssh\sama7g54 -o IdentitiesOnly=yes
wheels\libstdc++.so.6 root@192.168.100.46:/usr/lib/libstdc++.so.6
```

Step 19 — Install numpy on Board

```
pip3 install /data/numpy-1.24.4-cp38-cp38-linux_armv7l.whl
ldconfig
```

Create gnueabi symlinks (critical — board Python expects gnueabi, wheel is gnueabihf):

```
find /usr/lib/python3.8/site-packages/numpy -name "*.cpython-38-arm-linux-
gnueabihf.so" | while read f; do
    ln -sf "$f" "${f/gnueabihf/gnueabi}"
done
```

Verify:

```
python3 -c "import numpy, tflite_runtime; print('numpy:', numpy.__version__, '|
tflite:', tflite_runtime.__version__)"
# Expected: numpy: 1.24.4 | tflite: 2.11.0
```

> ⚠ If numpy imports with 'No module named numpy.core._multiarray_umath', the symlink step was missed or incomplete. Re-run the find/ln command.

5. Transfer Models & Run Inference

Step 20 — Transfer Model Files

Transfer Gaussian parameters and TFLite models from PC to board:

```
# Gaussian parameters (8 files)
for id in id_00 id_02 id_04 id_06; do
  scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes models/gaussian_mu_$id.npy
root@192.168.100.46:/data/
  scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes
models/gaussian_sigma_inv_$id.npy root@192.168.100.46:/data/
done

# TFLite models (4 files)
for id in id_00 id_02 id_04 id_06; do
  scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes models/hitachi_$id.tflite
root@192.168.100.46:/data/
done
```

Step 21 — Transfer Inference Scripts

```
scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes board/infer_gaussian.py
root@192.168.100.46:/data/
scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes board/infer_hitachi.py
root@192.168.100.46:/data/
```

Step 22 — Transfer Test Audio Files

On PC, generate test file lists (ensures no overlap with training data):

```
python -c "
from pc.data_loader import get_file_lists, IDS
import os, zipfile
for mid in IDS:
    _, test_norm, test_abn = get_file_lists(mid)
    with zipfile.ZipFile(f'test_{mid}.zip', 'w') as z:
        for f in test_norm[:30]:
            z.write(f, f'{mid}/normal/{os.path.basename(f)}')
        for f in test_abn[:30]:
            z.write(f, f'{mid}/abnormal/{os.path.basename(f)}')
"
```

Transfer and extract on board:

```
for id in id_00 id_02 id_04 id_06; do
  scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes test_$id.zip
root@192.168.100.46:/data/
done

# On board:
mkdir -p /data/mimii_test
for id in id_00 id_02 id_04 id_06; do
```

```
  python3 -c "import zipfile;
zipfile.ZipFile('/data/test_$id.zip').extractall('/data/mimii_test/')"
done
```

Step 23 — Run Inference

On the board:

```
# Gaussian model
python3 /data/infer_gaussian.py

# Hitachi TFLite model
python3 /data/infer_hitachi.py
```

6. Enable On-Board Microphones (Optional)

Step 24 — Close Jumper J3

Physically close jumper J3 on the SAMA7G54-EK board to enable power to the 4 PDMC MEMS microphones (Knowles SPK0641HT4H-1).

> ⚠ Without closing J3, the microphones receive no power and arecord will fail with 'BEs uses 0 channels' error.

Step 25 — Apply Device Tree Overlay

Reboot the board and interrupt U-Boot by pressing ENTER when prompted. Then:

```
edit bootcmd
```

Change from:

```
fatload mmc 1:1 0x63000000 sama7g5ek.itb; bootm 0x63000000#kernel_dtb
```

To:

```
fatload mmc 1:1 0x63000000 sama7g5ek.itb; bootm 0x63000000#kernel_dtb#pdmc0
```

Save and boot:

```
saveenv
boot
```

Step 26 — Record Audio

After applying the overlay, record 10 seconds from the 4 PDMC microphones:

```
arecord -D hw:0,0 -f S16_LE -r 16000 -c 4 -d 10 /data/recording.wav
```

7. Common Issues & Solutions

Issue	Cause	Fix
pyenv not found after install	PATH not updated	Manually add pyenv/bin and pyenv/shims to user PATH
venv uses Python 3.12 not 3.8	pyenv shims not active in venv creation	Use full path: .pyenv/versions/3.8.10/python.exe -m venv venv
TF import error: Descriptors cannot be created	protobuf version too new	pip install protobuf==3.20.3
pip install fails with SSL error on board	Board clock is wrong (year 2018)	date -s '2026-03-24 09:00:00' before any pip command
numpy build fails on board	No gcc toolchain on Yocto image	Use --no-deps flag; install pre-built wheel from Docker
numpy imports but crashes with multiarray error	gnueabihf vs gnueabi mismatch	Run the symlink creation command (Step 19)
glibc version mismatch when loading numpy .so	Wheel built against glibc 2.34, board has 2.31	Use python:3.8-bullseye Docker image (not slim/bookworm)
tflite set_tensor dimension mismatch	Stale inference script on board	Re-transfer infer_hitachi.py and confirm frame shape is (1,320)
arecord: BEs uses 0 channels	PDMC not routed to ASRC / J3 not closed	Close jumper J3 AND apply pdmc0 device tree overlay
SCP connection drops mid-transfer	SSH keepalive timeout	Use per-file zip archives transferred individually
Root filesystem full (0 bytes)	Linux4SAM root partition is only 500MB	Use /data partition on 32GB SD card for all files
unzip segfaults on board	BusyBox unzip can't handle large zips	Use Python: python3 -c "import zipfile; zipfile.ZipFile('x.zip').extractall('dir/')"
SSH still asks for password after key setup	Missing -o IdentitiesOnly=yes	Always use: scp -O -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes
WAV files unreadable by Python wave module	MIMII uses WAVE_FORMAT_EXTENSIBLE (format 65534)	Use the custom struct-based load_wav() in the board scripts

8. Results

8.1 PC Results

ID	Gaussian AUC	G Time (ms)	Hitachi AUC	H Time (ms)
id_00	0.9348	~17	0.7712	~37
id_02	0.9977	~17	0.9882	~43
id_04	0.9767	~21	0.9412	~42
id_06	0.9999	~20	0.9947	~43
Macro avg	0.9773	~19	0.9238	~41

8.2 Board Results (Approximate)

Model	Macro AUC	Avg Time (ms/file)	
Gaussian (board)	0.9472	~145	
Hitachi TFLite (board)	0.9833	~1025	

> ⓘ AUC values should match PC results closely. Inference time will be 10-15x slower due to ARM Cortex-A7 @ 800MHz vs modern x86 CPU.

9. Quick Reference

Board IP & Access

```
Board IP:     192.168.100.46
SSH:          ssh -i ~/.ssh/sama7g54 -o IdentitiesOnly=yes root@192.168.100.46
Serial:       PuTTY → Serial → COMx → 115200 baud
Login:        root (no password)
```

Key Paths

Location	Path
PC project	C:\Users\<user>\mimii_project\
PC models	C:\Users\<user>\mimii_project\models\
Board working dir	/data/
Board test audio	/data/mimii_test/
Board inference scripts	/data/infer_gaussian.py, /data/infer_hitachi.py

Key Commands

Task	Command
Fix board clock	date -s "2026-03-24 09:00:00"
Get board IP	udhcpc -i eth0 && ip addr show eth0
Check disk space	df -h
Check Python packages	python3 -c "import numpy, tflite_runtime; print('ok')"
Run Gaussian inference	python3 /data/infer_gaussian.py
Run Hitachi inference	python3 /data/infer_hitachi.py
Record audio (after J3+overlay)	arecord -D hw:0,0 -f S16_LE -r 16000 -c 4 -d 10 /data/rec.wav